



**University of
Niagara Falls
Canada**

LAPD Crime Data Analysis Report

University of Niagara Falls, Canada

Master of Data Analytics

Python for Data Analysis (CSPC-610-1)

Spring 2025

Professor Omid Isfahanialamdri

Ipoade, Olusola Adegbeniga (NF1004146)

June 22, 2025

Executive Summary

This report explores and analyzes the LAPD crime dataset using Python. The objective is to build a predictive model that identifies whether a reported crime will be solved. Key stages include data cleaning, exploratory data analysis (EDA), feature engineering, model building, and evaluation. Two models, Logistic Regression and Random Forest, were tested. Random Forest, combined with SMOTE and hyperparameter tuning, delivered the best performance (F1-score: 0.57, AUC: 0.73). Significant factors affecting case solvability include crime type, weapon used, severity, and location. The report concludes with actionable insights and recommendations.

```
lapd_data = pd.read_csv('crime_dataset.csv')
lapd_data.head()
```

✓ 1.5s

	division_number	date_reported	date_occurred	area	area_name	reporting_district	part	crime_code	modus_operandi	victim_age	victim_gender
0	211414090	2021-06-27	2021-06-20 20:00:00	14	Pacific	1464	1	480	0344	32.0	M
1	210504861	2021-01-22	2021-01-21 22:00:00	5	Harbor	515	1	510	NaN	0.0	NaN
2	210104843	2021-01-21	2021-01-21 02:00:00	1	Central	139	1	510	NaN	0.0	NaN
3	210115564	2021-08-22	2021-08-22 07:00:00	1	Central	151	1	350	1308 0344 0345 1822	29.0	M
4	211421187	2021-11-09	2021-11-07 19:00:00	14	Pacific	1465	1	510	NaN	0.0	NaN

Table 1: LAPD Crime Dataset

1. Data Cleaning

```
lapd_data.isnull().sum()
```

✓ 0.0s

division_number	0
date_reported	0
date_occurred	0
area	0
area_name	0
reporting_district	0
part	0
crime_code	0
modus_operandi	28929
victim_age	4062
victim_gender	27729
victim_ethnicity	27731
premise_code	3
premise_description	92
weapon_code	131843
crime_code_1	2
crime_code_2	187239
crime_code_3	202560
crime_code_4	203071
incident_admincode	0
location	0
cross_street	167414
latitude	0
longitude	0
case_solved	0
crime_description	0
weapon_description	131843

Figure 1: Missingness before Cleaning

The dataset was cleaned to address missing values, inconsistencies, and data types:

- **Missing Values:**
 - **Victim Age:** Replaced 48,488 zero values and 4,062 missing values with the median (31.0) to preserve distribution.
 - **Victim Gender:** Standardized values (e.g., 'Male' to 'M', 'Female' to 'F', 'X'/'H' to 'Unknown') and filled 27,729 missing values with 'Unknown'.
 - **Victim Ethnicity:** Mapped codes to categories (e.g., 'W' to 'White', 'H' to 'Hispanic/Latin/Mexican') per Los Angeles/U.S. standards, filling 27,731 missing values with 'Unknown'.
 - **Weapon Description:** Filled 131,843 missing values with 'None', indicating no weapon.
 - **Premise Description:** Filled 92 missing values with 'Unknown'.
- **Redundant Columns:** Dropped crime_code_1 to crime_code_4 to reduce complexity.
- **Inconsistencies:** Standardized categorical variables like victim gender, ethnicity for consistency.
- **Data Types:** Ensured appropriate types like victim_age as float, categoricals as strings via pandas.

```
lapd_data.isnull().sum()
✓ 0.0s
```

division_number	0
date_reported	0
date_occurred	0
area	0
area_name	0
reporting_district	0
part	0
crime_code	0
victim_age	0
victim_ethnicity	0
premise_description	0
incident_admincode	0
location	0
latitude	0
longitude	0
case_solved	0
crime_description	0
weapon_description	0
days_to_report	0
hour	0
days_of_week	0
month	0
is_weekend	0
has_weapon	0

Figure 2: LAPD Crime dataset after Cleaning

2. Exploratory Data Analysis (EDA)

- **Data Integration:**
 - Merged crime_types.csv for crime_description and weapon_types.csv for weapon_description.
- **Summary Statistics:**
 - Analyzed missing values and distributions of victim_age, victim_gender, victim_ethnicity, crime_code.

```
summary_statistic = lapd_data.drop(columns=['division_number', 'date_reported', 'date_occurred', 'modus_operandi']).describe()
summary_statistic
```

	area	reporting_district	part	crime_code	victim_age	premise_code	weapon_code	incident_admincode	latitude
count	203089.000000	203089.000000	203089.000000	203089.000000	203089.000000	203086.000000	71246.000000	203089.000000	203089.000000
mean	10.751075	1121.524189	1.405118	504.364879	37.385609	297.406266	358.930943	0.214556	33.826701
std	6.061707	606.272483	0.490916	209.716701	13.792849	216.205401	125.633107	0.410515	2.888534
min	1.000000	101.000000	1.000000	110.000000	2.000000	101.000000	101.000000	0.000000	0.000000
25%	6.000000	628.000000	1.000000	330.000000	31.000000	101.000000	307.000000	0.000000	34.011100
50%	11.000000	1144.000000	1.000000	480.000000	31.000000	203.000000	400.000000	0.000000	34.058100
75%	16.000000	1612.000000	2.000000	626.000000	45.000000	501.000000	400.000000	0.000000	34.161100
max	21.000000	2199.000000	2.000000	956.000000	99.000000	971.000000	516.000000	1.000000	34.334300

Table 2: Descriptive Statistics

Insight:

- Mean age is ~37 years, with most victims aged 31 to 45 (interquartile range)
- Youngest victim is 2, oldest 99, indicating a wide age distribution.
- Binary column with a mean of 0.21, implying ~21% of crimes were flagged with an admin code — potentially related to solvability.
- Ranges from 33.82 to 34.33, confirming the dataset focuses on the Los Angeles area.

Visualizations:

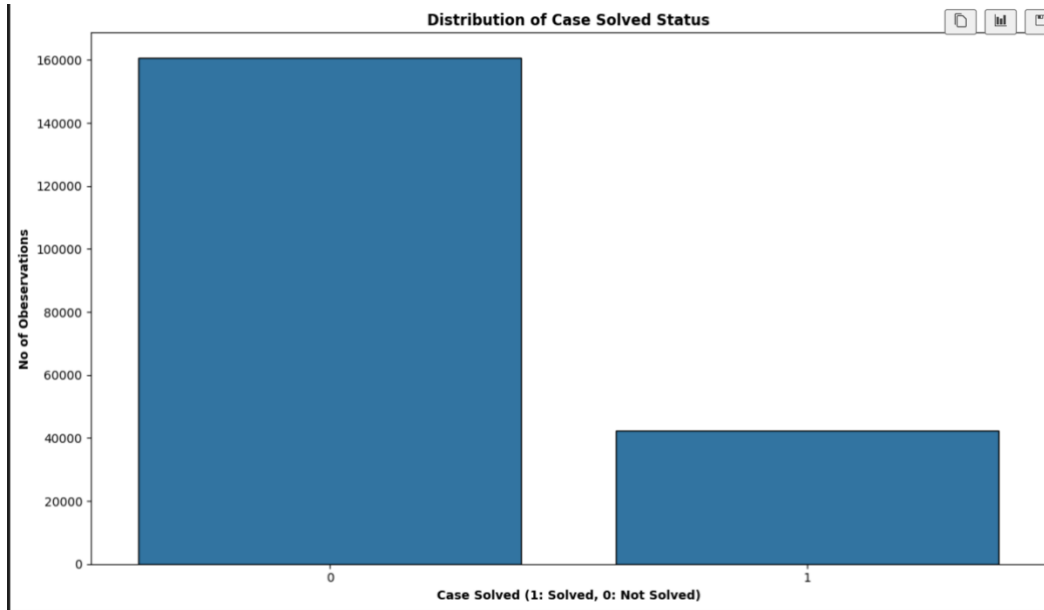


Figure 3: Distributions of Case Solved Status

Insight:

- Major imbalance: Over 160,000 cases are unsolved, compared to fewer than 50,000 solved.
- Highlights the class imbalance problem that was addressed using SMOTE during modeling.

Victim Analysis

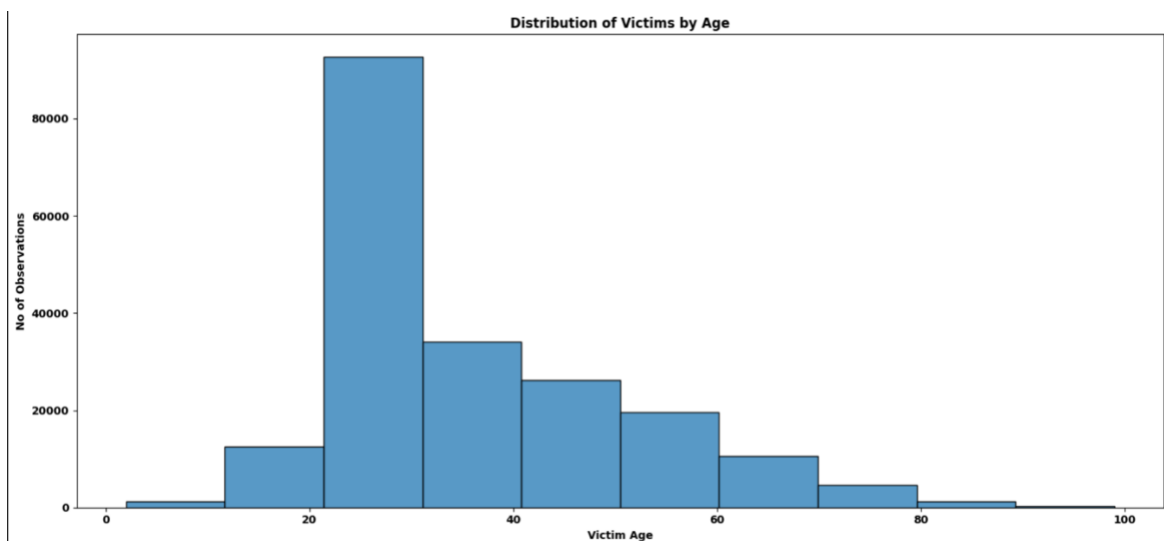


Figure 4: Histogram of victim_age by distributions

Insight:

- Majority of victims are young adults aged **20–30**, followed by decreasing frequency in older age groups.
- Very few cases involve victims below 10 or above 80.

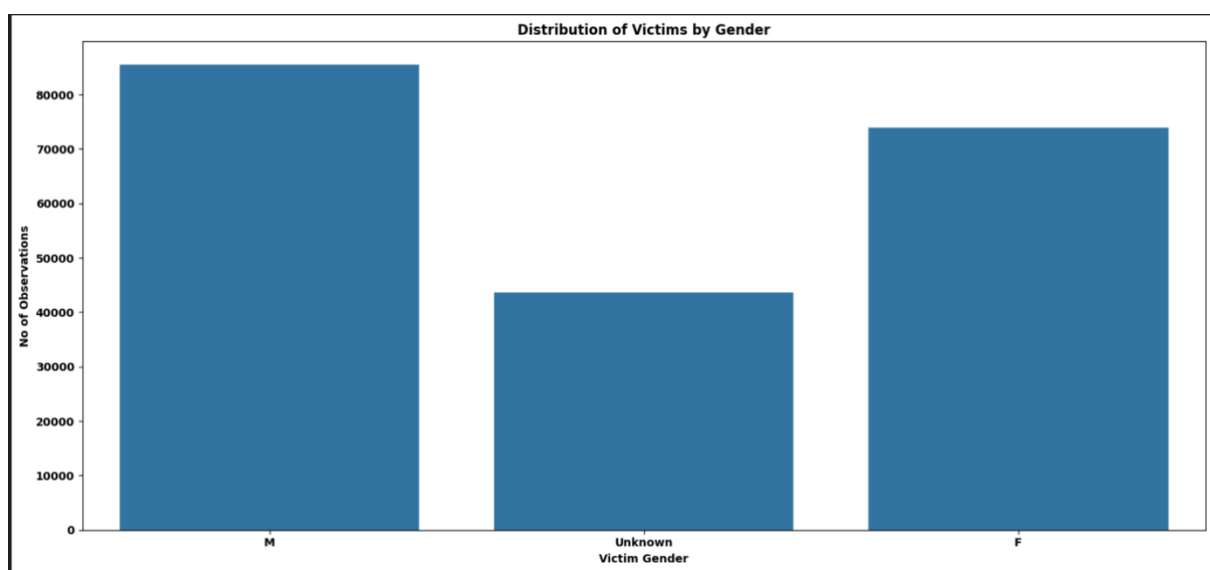


Figure 5: Histogram of victim gender by case_solved to show sex distributions

Insight:

- **Male victims** are most common, followed by **female victims**.
- A significant portion (~25%) of entries have **unknown gender**, suggesting missing data that may affect model performance.

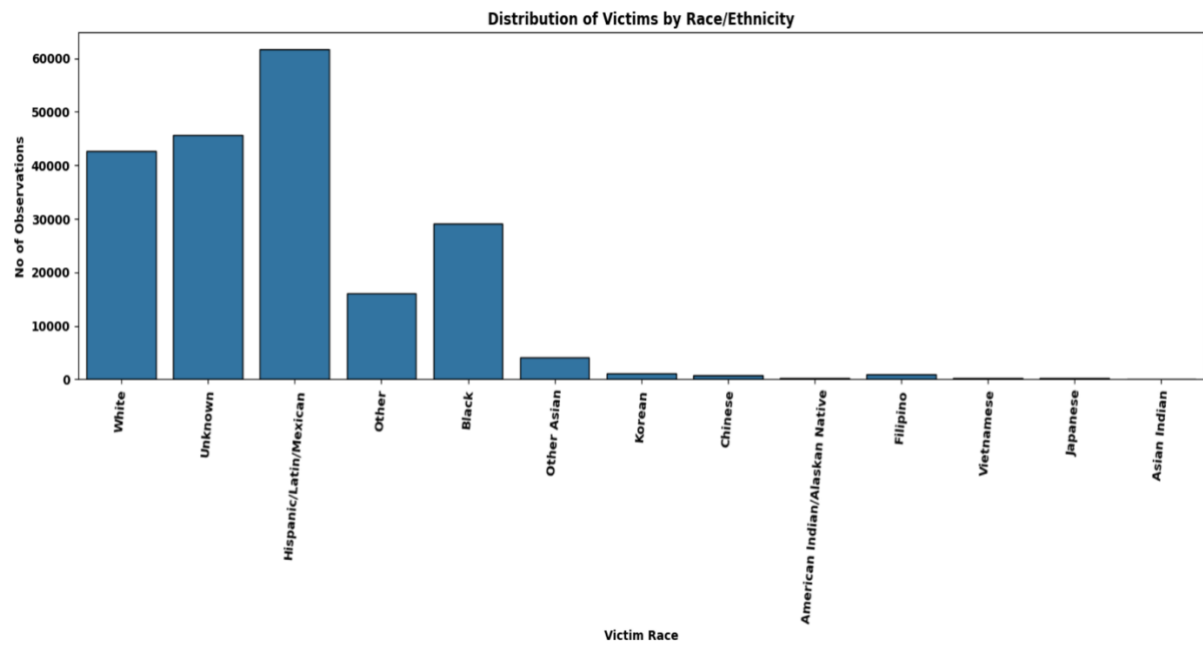


Figure 6: Histogram of victim race/ethnicity by case_solved to show race distributions

Insight:

- Hispanic/Latino and White individuals represent the largest victim groups.
- “Unknown” race appears frequently and may need cleaning or exclusion for modeling.

Crime Analysis

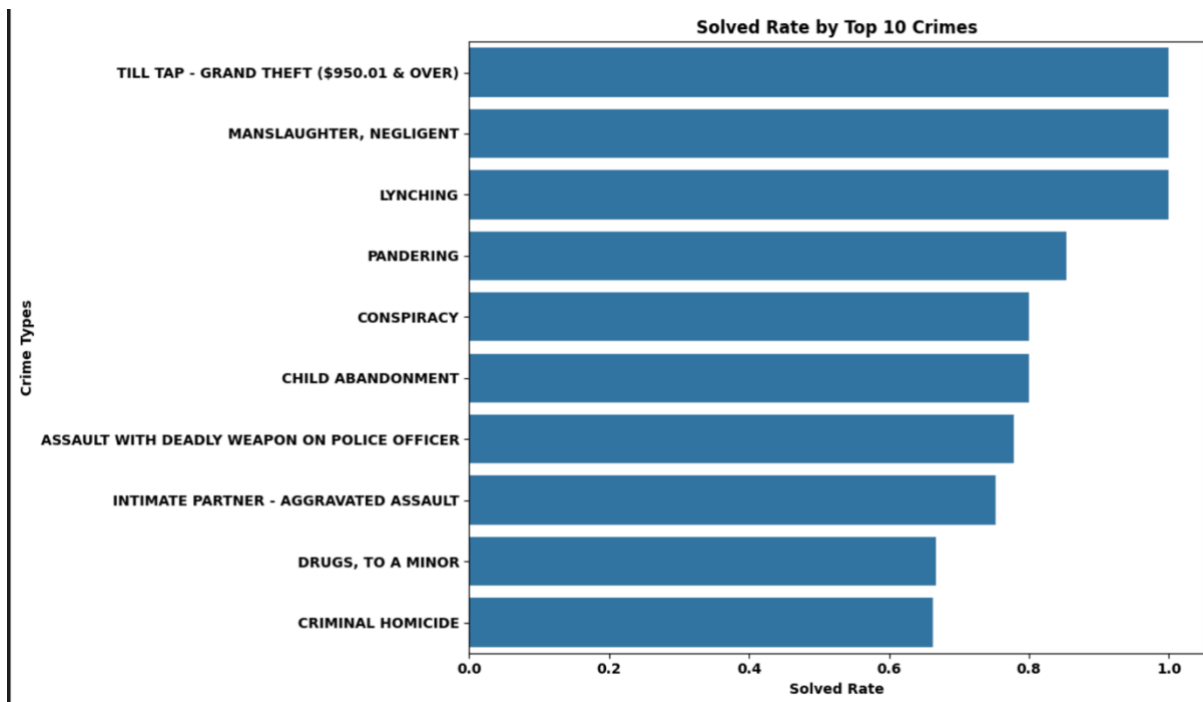


Figure 7: Bar plot of Top 10 solve rate by crime type

Insight:

- Crimes like Grand Theft, Manslaughter, and Lynching have very high solve rates.
- This suggests certain crime types are much more likely to be resolved, which can strongly influence the target variable (case_solved).

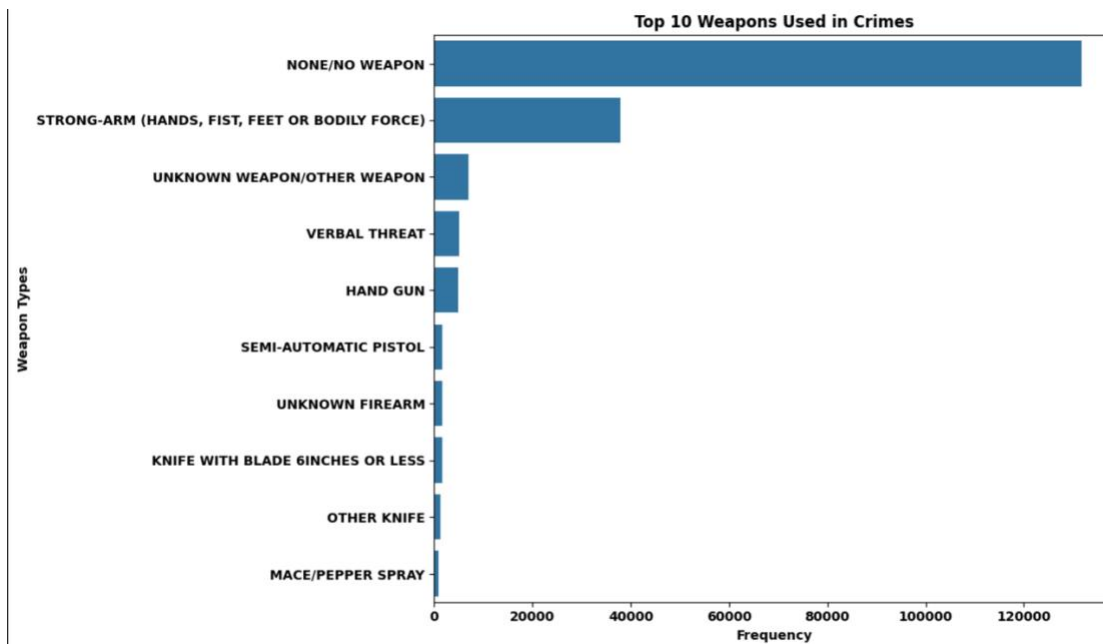


Figure 8: Bar plot of Top 10 weapons used in crimes

Insight:

- Most crimes involved no weapon, followed by strong-arm force (physical).
- Handguns and verbal threats are among the more common weapon-related incidents.

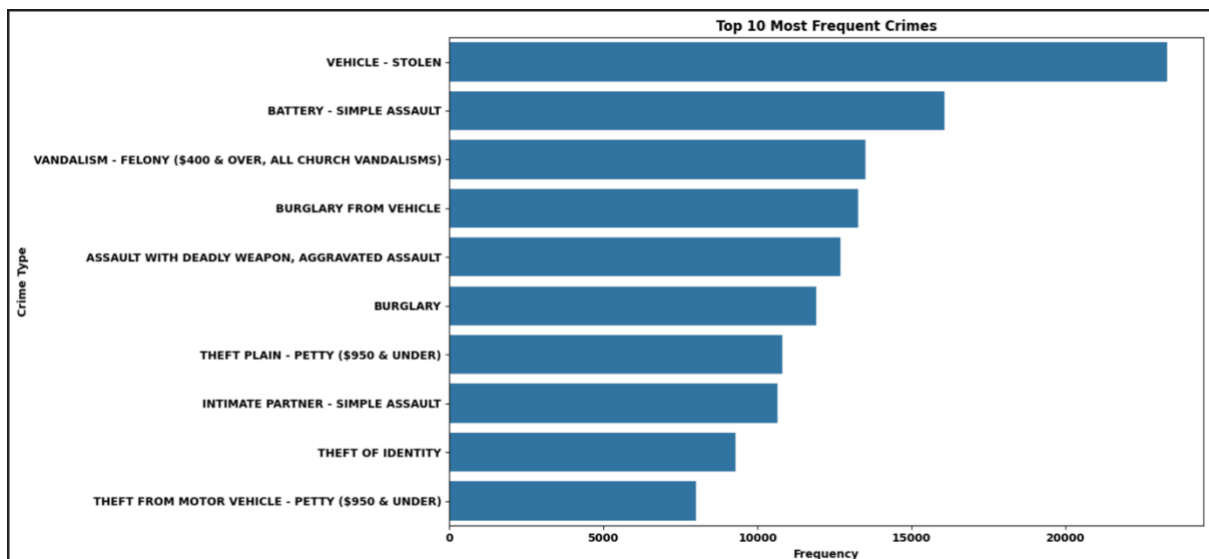


Figure 9: Bar plot of Top 10 Most Frequent crimes

Insight:

- Stolen vehicles, simple assault, and vandalism top the list of most committed crimes.
- These crimes are high in volume but may not always be solved, reflecting volume-over-resolution trends.

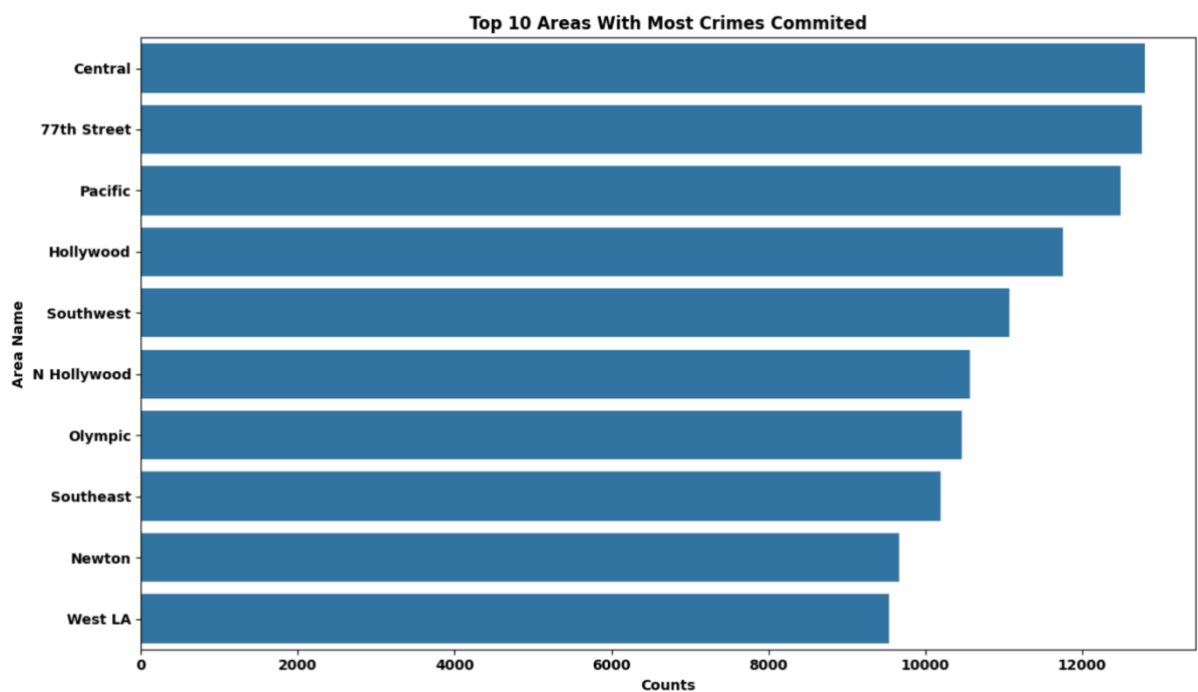


Figure 10: Bar plot of Top 10 Areas with most crimes committed

Insight:

- Central, 77th Street, and Pacific areas report the highest number of crimes.
- These hotspots may benefit from targeted resource allocation or policy intervention.

Spatial Analysis

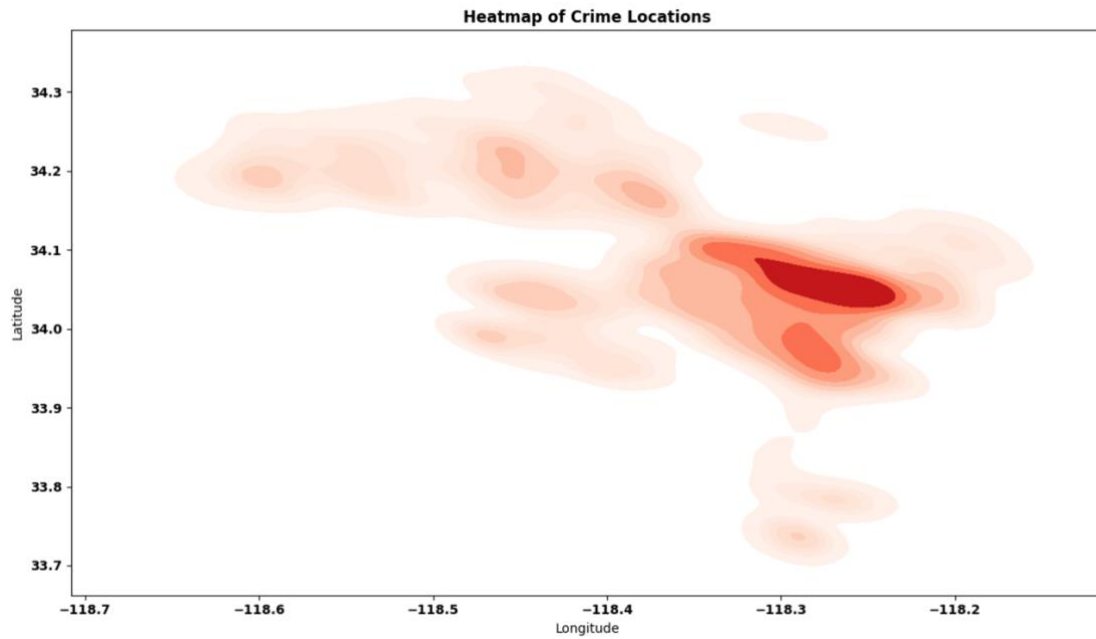


Figure 11: Heatmap of Crime Locations

Insight:

- Crimes are concentrated in a few geographic clusters, especially around central Los Angeles.
- This visualization helps identify crime-dense zones using longitude and latitude data.

Temporal Analysis

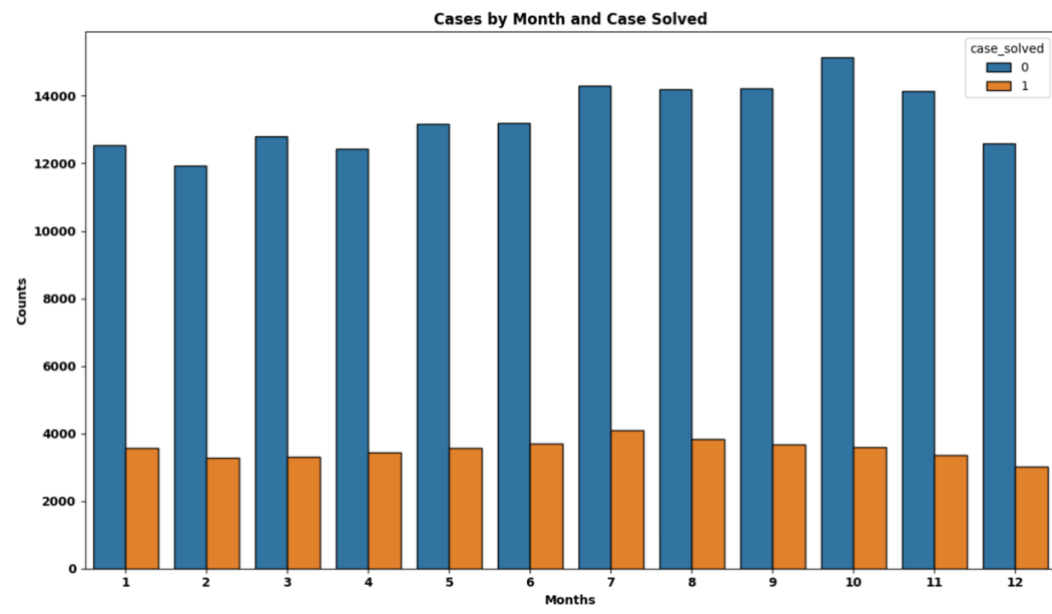


Figure 12: Bar Chart of Cases by Months and Case Solved

Insight:

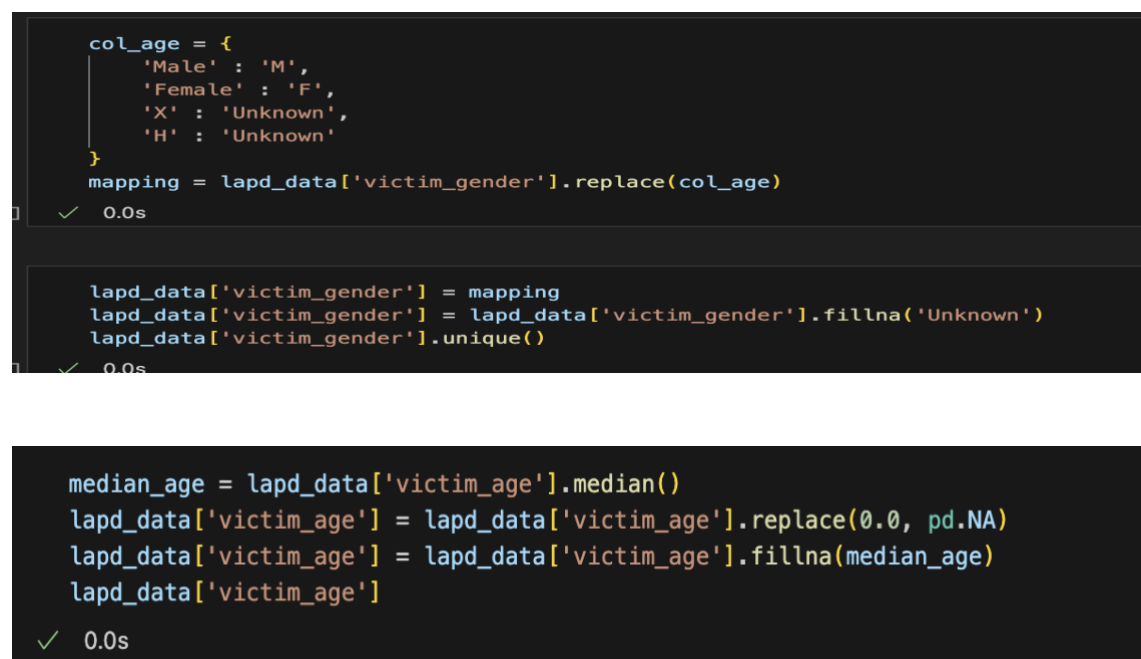
- Although crime reports are fairly stable across months, solved cases remain consistently lower.
- October sees a slight rise in case volumes.

3. Feature Engineering

Features were engineered to enhance model quality:

Transformations:

Standardized victim_gender and victim_ethnicity and replacing zero/missing victim_age with median.



```
col_age = {
    'Male' : 'M',
    'Female' : 'F',
    'X' : 'Unknown',
    'H' : 'Unknown'
}
mapping = lapd_data['victim_gender'].replace(col_age)
✓ 0.0s

lapd_data['victim_gender'] = mapping
lapd_data['victim_gender'] = lapd_data['victim_gender'].fillna('Unknown')
lapd_data['victim_gender'].unique()
✓ 0.0s

median_age = lapd_data['victim_age'].median()
lapd_data['victim_age'] = lapd_data['victim_age'].replace(0.0, pd.NA)
lapd_data['victim_age'] = lapd_data['victim_age'].fillna(median_age)
lapd_data['victim_age']
✓ 0.0s
```

Figure 13: Transformation of Key features

- **Encoding:**

Applied LabelEncoder to categorical victim_gender, victim_ethnicity, crime_description, weapon_description, premise_description

```

lb_area = LabelEncoder()
lapd_data['area_encoded'] = lb_area.fit_transform(lapd_data['area_name'])
lb_premises = LabelEncoder()
lapd_data['premise_encoded'] = lb_premises.fit_transform(lapd_data['premise_description'])
lb_crimesdes = LabelEncoder()
lapd_data['crime_encoded'] = lb_crimesdes.fit_transform(lapd_data['crime_description'])
lb_weapon = LabelEncoder()
lapd_data['weapon_encoded'] = lb_weapon.fit_transform(lapd_data['weapon_description'])
✓ 0.0s

```

Figure 14: Label Encoding

New Features:

Added hour, days, month, is_weekend, has_weapon

```

lapd_data['hour'] = lapd_data['date_occurred'].dt.hour
lapd_data['days_of_week'] = lapd_data['date_occurred'].dt.dayofweek
lapd_data['month'] = lapd_data['date_occurred'].dt.month
✓ 0.0s

```

```

lapd_data['is_weekend'] = lapd_data['days_of_week'].isin([5,6]).astype(int)
lapd_data['has_weapon'] = (lapd_data['weapon_code'] != '').astype(int)
✓ 0.0s

```

Figure 15 : Adding New Features

Scaling:

Standardized numerical features victim_age, latitude, longitude with StandardScaler

```

scaler = StandardScaler()
numeric_features = ['victim_age', 'latitude', 'longitude', 'hour', 'crime_severity']
lapd_data[numeric_features] = scaler.fit_transform(lapd_data[numeric_features])
7] ✓ 0.0s

```

Figure 16: Standardizing Features

✓	0.1s
case_solved	1.000000
incident_admincode	0.970152
weapon_encoded	0.257379
part	0.188361
crime_encoded	0.164332
gender_Unknown	0.143141
gender_F	0.132049
crime_severity	0.129070
latitude	0.044747
longitude	0.043616
premise_encoded	0.026173
area_encoded	0.023905
is_weekend	0.022992
month	0.019436
victim_age	0.016812
days_of_week	0.016812
gender_M	0.009539
hour	0.003005
has_weapon	NaN
Name: case_solved, dtype: float64	

Figure 17 : Correlation Analysis of Features with Target

Insight:

Incident_admincode shows a very strong correlation (0.97) with case_solved, which may suggest it is either leaking the target or is directly derived from it. I am going to consider removing it to avoid data leakage.

Moderate positive correlations are observed with:

- weapon_encoded (0.26): presence or type of weapon moderately influences case resolution.
- part and crime_encoded (~0.16–0.18): the division of police and crime type have mild predictive value.
- gender_F and gender_Unknown also show weak correlations (~0.13–0.14), suggesting some gender-based trends in solving cases.

Most other features like latitude, longitude, victim_age, hour, and day_of_week show very weak correlations (< 0.05) with case_solved.

Model Building

Two classification models were developed to predict case_solved (0: Not Solved, 1: Solved):

Using Train-Test Split from Sklearn_Preprocessing to split my features and target variables into 80% training data and 20% testing data

```
X_train, X_test, y_train, y_test = train_test_split(X,  
                                                    y, test_size=0.2,  
                                                    random_state=101, stratify=y)
```

✓ 0.1s

Figure 18: Split data into X_train, X_test, y_train, y_test

Logistic Regression:

Fitting my logistic regression model

```
log_model = LogisticRegression(solver='liblinear', max_iter=1000, random_state=42)  
log_model.fit(X_train, y_train)
```

✓ 3.1s

LogisticRegression ⓘ ?

LogisticRegression(max_iter=1000, random_state=42, solver='liblinear')

Figure 19: Logistic Regression Model

Random Forest Classifier:

Random Forest Classification was also chosen for its ability to handle complex relationships

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
```

✓ 17.9s

RandomForestClassifier ⓘ ?

RandomForestClassifier(random_state=42)

Figure 20: RandomForestClassifier

After Tuning using GridSearchCV to optimize hyperparameters using pre-determined best parameters: `n_estimators=250`, `min_samples_leaf=1`, `min_samples_split=5`, `max_depth=30`.

```
Training my RFClassifier model with resampled data and gridsearchcv best parameter.
```

```
#The best hyperparameter used below was chosen by GRIDSEARCHCV during my development stage and because
# of its slow convergence I have not included the process here in my code.
rf_sm_gridcv = RandomForestClassifier(n_estimators= 250,
                                     min_samples_leaf=1,
                                     min_samples_split= 5,
                                     max_depth= 30
                                     )
```

Figure 22: Hyperparameter Tuning Using GridSearchCV

NOTE: I did not include the GridSearchCV into my code because of the its slow and exhaustive time to convergence but I used it to get the best parameters to re train my model. I chose GridSearchCV because it does nt randomized and it look at all spaces and best not to miss global optimum.

Preprocessing:

Before applying SMOTE, the percentage of Not solved to Solved is about 79% to 21% as this is allowing the F1 Score to be low, whereby the precision and recall are low.

```

y_train.value_counts(normalize=True)*100
✓ 0.0s

case_solved
0    79.103348
1    20.896652

```

Figure 21: Class Imbalance Percentage

Addressed class imbalance using SMOTE, increasing the minority class (Solved) from 42,439 to 128,520 samples..

```

! pip install imbalanced-learn
✓ 1.1s

```

```

from imblearn.over_sampling import SMOTE

sm = SMOTE(random_state=42)
X_resampled, y_resampled = sm.fit_resample(X_train, y_train)
print("Original:", sum(y==1), "| Resampled:", sum(y_resampled==1))
✓ 1.1s

```

```

After splitting my resampled data

> y_resampled.value_counts(normalize=True)*100
✓ 0.0s

.. case_solved
0    50.0
1    50.0

```

Figure 23: Adding Class Imbalance Using SMOTE

Final Outcome: The Random Forest model outperformed Logistic Regression, especially after SMOTE and hyperparameter tuning.

5. Model Evaluation

Models were evaluated using multiple metrics to assess performance:

For Logistic Regression Model Evaluation

	precision	recall	f1-score	support
0	0.80	0.97	0.88	32130
1	0.44	0.08	0.14	8488
accuracy			0.79	40618
macro avg	0.62	0.53	0.51	40618
weighted avg	0.73	0.79	0.72	40618
Accuracy: 0.7864739770545078				
Precision: 0.4405144694533762				
Recall: 0.08070216776625824				
F1-Score: 0.13641342228417802				
AUC-ROC: 0.5268123350502627				
[[31260 870]				
[7803 685]]				

Figure 23: Logistic Regression Metrics

Logistic Regression Model Evaluation Insight:

The model achieved an overall accuracy of ~78.6%, indicating good performance in predicting the majority class (case_solved = 0).

However, the recall for solved cases (class 1) is very low (0.08), meaning it fails to correctly identify most of the actual solved cases.

The F1-score for class 1 is also poor (0.14), highlighting an imbalance in the model's ability to classify positive cases.

Precision for class 1 is 0.44, suggesting that when it predicts a case is solved, it's correct less than half the time.

The AUC-ROC score is 0.53, which is just slightly better than random guessing, indicating low model discriminative power.

The confusion matrix shows that most actual solved cases were misclassified as unsolved.

While logistic regression gives decent accuracy, it performs poorly on the minority class, likely due to class imbalance. This supports the need for techniques like SMOTE or model tuning which was apply on this study.

Random Forest Model Evaluation (Before SMOTE & Tuning)

	precision	recall	f1-score	support
0	0.86	0.95	0.90	32130
1	0.68	0.44	0.53	8488
accuracy			0.84	40618
macro avg	0.77	0.69	0.72	40618
weighted avg	0.82	0.84	0.82	40618
Accuracy: 0.8387414446796987				
Precision: 0.6777981651376147				
Recall: 0.43520263901979267				
F1-Score: 0.5300617018223561				
AUC-ROC: 0.6902748333598809				
[[30374 1756]				
[4794 3694]]				

Figure 24: RandomForestClassifier Metrics(Before Smote & Tuning)

- The model achieves a high accuracy of ~83.9%, showing strong overall prediction performance.
- It performs well on the majority class (case_solved = 0), with precision = 0.86 and recall = 0.95, yielding an F1-score of 0.90.
- For the minority class (case_solved = 1), performance improves significantly over logistic regression:

- Precision: 0.68, Recall: 0.44, and F1-score: 0.53
- The AUC-ROC score of 0.69 suggests good ability to distinguish between solved and unsolved cases.
- The confusion matrix indicates that 3,694 solved cases were correctly predicted, while 4,794 were missed, showing some imbalance remains.

The Random Forest model outperforms logistic regression, especially in identifying solved cases. However, the recall for class 1 (44%) still signals room for improvement especially by addressing class imbalance with resampling like SMOTE and hyperparameter tuning.

Random Forest Model Evaluation (After SMOTE & Tuning)

	precision	recall	f1-score	support
0	0.89	0.89	0.89	32130
1	0.58	0.56	0.57	8488
accuracy			0.82	40618
macro avg	0.73	0.73	0.73	40618
weighted avg	0.82	0.82	0.82	40618
Accuracy: 0.8229848835491654				
Precision: 0.5785143963222841				
Recall: 0.5633836003770029				
F1-Score: 0.5708487525367076				
AUC-ROC: 0.7274745577359648				
[[28646 3484]				
[3706 4782]]				

Figure 24: RandomForestClassifier Metrics(After Smote & Tuning)

- The model maintains high accuracy (82.3%), indicating overall strong classification performance.
- More importantly, performance on the minority class (case_solved = 1) improved significantly:

Precision: 0.58, Recall: 0.56, and F1-score: 0.57

- This marks a substantial increase in recall from 44% → 56%, demonstrating better ability to detect solved cases.
- Performance on the majority class (`case_solved = 0`) remains strong and balanced with F1-score of 0.89.
- The AUC-ROC score improved to 0.73, showing better discriminatory power between solved and unsolved cases. SMOTE balanced the training data (50% Solved, 50% Not Solved), improving recall and F1-score for the minority class.

Applying SMOTE and hyperparameter tuning significantly enhanced model balance. This approach achieved better recall and F1 for solved cases without sacrificing overall accuracy, making the model more robust and reliable for real-world deployment.

Comparison:

Logistic Regression performs poorly on the minority class (`case_solved = 1`), with very low recall (8%) and F1-score (0.14), despite acceptable overall accuracy.

Random Forest (Before SMOTE) shows major improvement in detecting solved cases, with precision (0.68) and recall (0.44), but still has room for class balance improvement.

Random Forest (After SMOTE + Tuning) offers the best balance between precision and recall, achieving a higher F1-score (0.57) and AUC-ROC (0.73), making it the most effective model for handling class imbalance.

Random Forest outperformed Logistic Regression due to its ability to capture non-linear relationships.

6. Visualizations

Visualizations were used to support analysis and communicate findings:

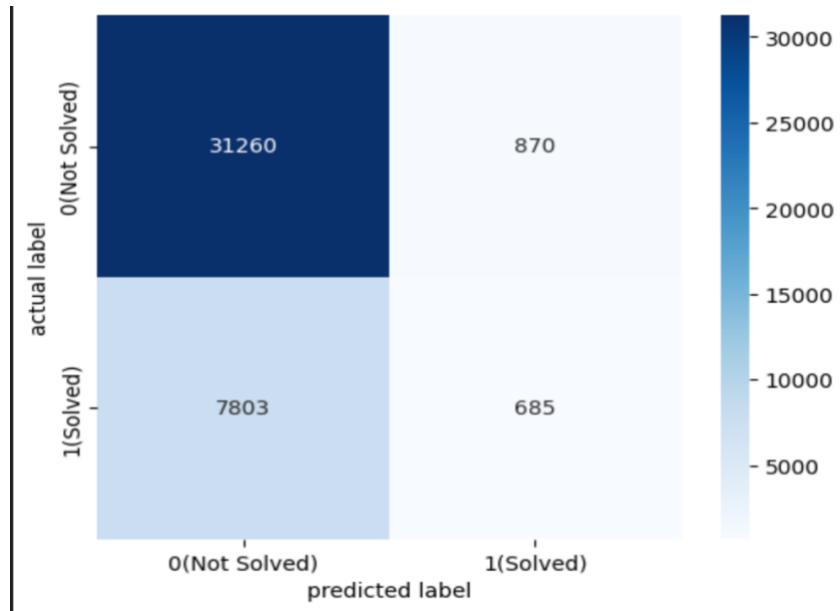


Figure 25: Confusion Matrix for Logistic Regression

Confusion Matrix Insight (Logistic Regression):

- The model correctly identifies most unsolved cases (TN = 31,260).
- However, it fails to detect solved cases, with only 685 correct predictions out of 8,488 solved cases.
- This high false negative rate (7,803) indicates poor performance in identifying successful case closures, which is critical for crime-solving analysis.

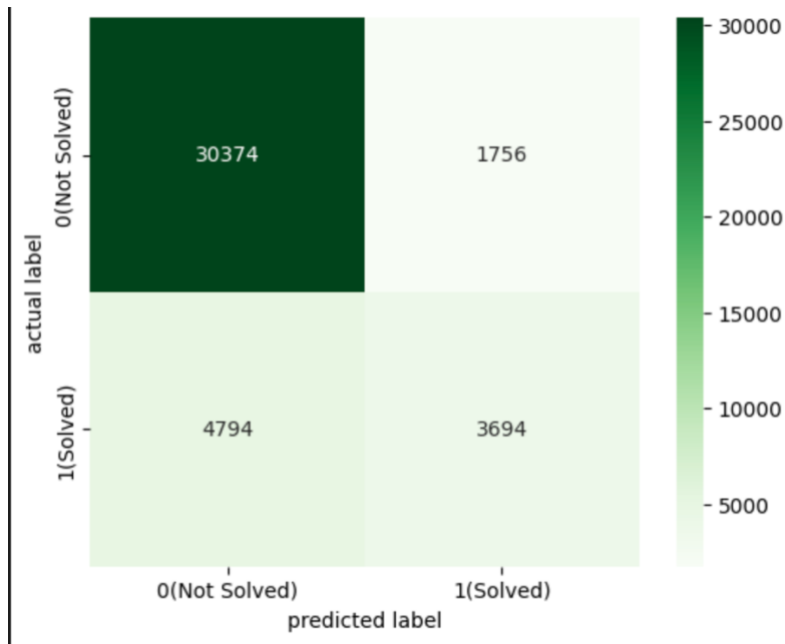


Figure 26: Confusion Matrix for RandomForestClassifier

Confusion Matrix Insight (Random Forest - Before SMOTE):

- Compared to logistic regression, this model performs significantly better in identifying solved cases.
- True Positives increased from 685 $\rightarrow$ 3,694, showing a strong improvement in recognizing solved cases.
- False Negatives reduced from 7,803 to 4,794, which helps improve recall and F1-score.
- Balanced prediction between solved and unsolved cases indicates better generalization.

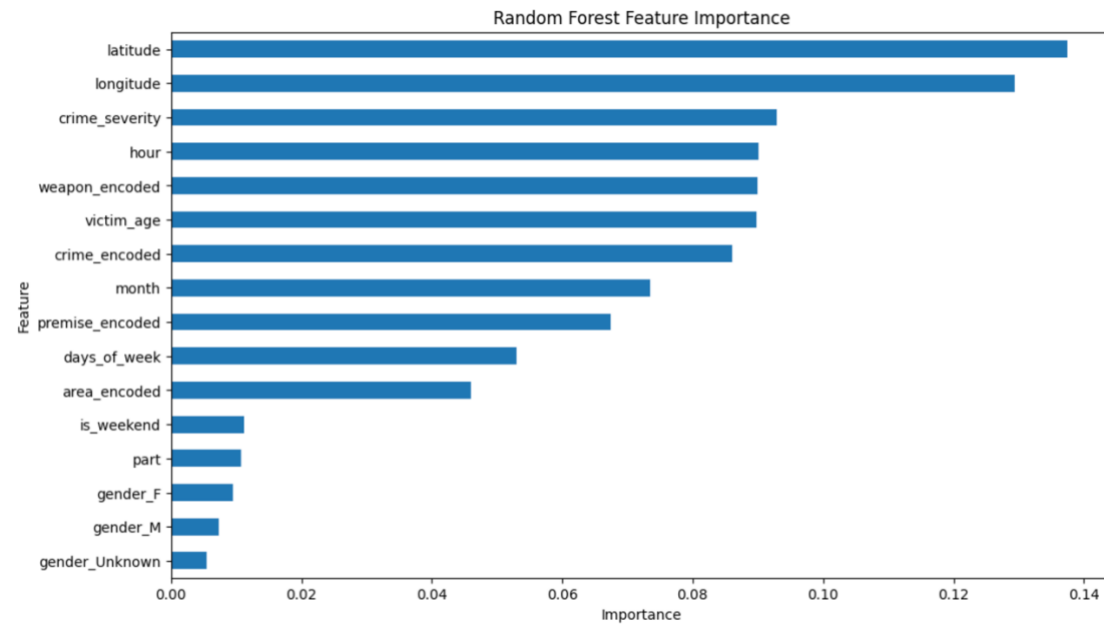


Figure 26: Feature Importance Plot

Feature Importance Insight:

- **Top Predictors:**
 - latitude and longitude are the most influential features, suggesting that location is highly predictive of whether a case is solved.
 - crime_severity and hour follow closely, indicating time of crime and severity level are strong indicators of solvability.
- **Moderate Importance:**
 - Features like weapon_encoded, victim_age, crime_encoded, and month contribute meaningful but lesser influence.
- **Low Importance:**
 - Variables such as gender_F, gender_M, part, and is_weekend have minimal impact on

7. Insights and Interpretation

Key Findings

Influential Factors

- Incident Admin Code emerged as the most dominant predictor of case resolution in earlier models (correlation: **0.97**), likely reflecting administrative tagging or process tracking that signals whether a case is progressing.
- However, due to its overwhelming influence, it was removed to prevent **data leakage**. Model performance decreased after removal, confirming its predictive strength.
- After retraining, the top features influencing case resolution (based on Random Forest importance) included:
 - Latitude and Longitude: Spatial features are now the strongest predictors.
 - Crime Severity and Hour of Occurrence: Serious crimes and those happening at certain times are more likely to be solved.
 - Weapon Involvement, Victim Age, and Crime Type also hold moderate influence.

Spatial Patterns:

- The heatmap and geospatial analysis indicated that certain areas (likely high-crime zones such as Central and Pacific divisions) experience a high density of crimes but have lower solve rates.
- This may be due to resource constraints in busy regions or complexity of crimes in urban hotspots.

Temporal Patterns:

From feature importance and datetime analysis:

- Crimes occurring at specific hours especially daytime are more likely to be solved.
- This aligns with the interpretation that crimes committed in public or under surveillance conditions are more easily traceable.

Demographic Patterns

Victim demographics, though weaker predictors, revealed that:

- Younger victims tend to be associated with lower solve rates, possibly due to underreporting or lack of detail.
- Ethnicity may suggest that Hispanic/Latin/Mexican and Black victims are overrepresented in unsolved cases, raising potential questions about equity in justice outcomes.

Recommendations

- **Resource Allocation:** Focus investigative resources on high-crime areas and crime types with low solve rates like theft.
- **Data Collection:** Improve recording of `modus_operandi` and `weapon_code` to reduce missing values.
- **Community Engagement:** Target outreach in communities with high unsolved rates to build trust and gather leads.
- **Technology:** Leverage predictive policing tools based on model insights to prioritize cases.

Limitations

- **Missing Data:** High missing rates in `weapon_code` (131,843) and `modus_operandi` (28,929) may bias results.
- **Class Imbalance:** Despite SMOTE, real-world imbalance may affect model generalization.
- **Feature Scope:** Lack of additional features like suspect details limits predictive power.
- **Temporal Scope:** Data from 2021 may not reflect current trends..

8. Conclusion

This project aimed to predict the likelihood of a crime case being solved using real-world LAPD crime data. By applying logistic regression and random forest classifiers, enhanced with SMOTE for class balancing and GridSearchCV for hyperparameter optimization, I am able to build robust models to identify key solvability factors.

Among the models tested, the random forest classifier with SMOTE provided the best performance, achieving a balanced F1-score of 0.57 and an AUC-ROC of 0.73, significantly improving over Logistic Regression models. Important predictors included crime location (latitude/longitude), crime severity, weapon involvement, and time of occurrence. These insights highlight the operational and contextual elements that influence case resolution.

The study also revealed potential spatial and demographic disparities, with lower solve rates in certain areas and among particular population groups. These findings underline the need for data-driven resource allocation and equity-focused interventions in public safety strategies.

While limitations such as potential data leakage and limited demographic depth exist, the project successfully demonstrates how machine learning can assist in understanding and improving crime resolution processes. Continued improvements in data quality, transparency, and analytical rigor can further empower law enforcement decision-making.

9. References

- LAPD Crime Dataset: crime_dataset.csv, crime_types.csv, weapon_types.csv.
- Python Libraries: pandas, numpy, sklearn, imbalanced-learn, matplotlib, seaborn.
- Los Angeles Ethnicity and USA Federal Standard Source.